Uncertainty-Aware Hybrid CNN Ensemble for Four-Class Acute Lymphoblastic Leukemia Subtyping with MC Dropout

How It Works — A Non-Technical Guide

Divya Reddy

Prepared for the client

IMPORTANT NOTICE: This software is a research and educational project. It is **not** approved for clinical diagnosis. All results must be verified by a qualified medical professional.

Generated from project results dated 2026-10-10 14:55 UTC

Table of Contents

1. What This Project Does
2. The Data
3. How a Computer Learns to Recognise Cells
4. The Three Backbones
5. The Hybrid Ensemble
6. Grad-CAM: Where the Model Is Looking
7. Results
8. The Web Application
9. Where Everything Is
10. How It Was Built
11. Installation
12. Using the App — Step-by-Step Demo
13. Glossary
14. References

1. What This Project Does

Acute Lymphoblastic Leukaemia (ALL) is a type of blood cancer that affects white blood cells. Doctors diagnose it by examining a thin smear of blood under a microscope — a **peripheral blood smear** (a glass slide coated with a drop of blood, spread thin, stained with dye and photographed). This project builds software that looks at those photographs and sorts them into one of four groups:

- **Benign** — normal, healthy cells (no cancer).
- **[Malignant] early Pre-B** — cancer cells at an early stage called "early Pre-B".
- **[Malignant] Pre-B** — cancer cells at the "Pre-B" stage.
- **[Malignant] Pro-B** — cancer cells at the "Pro-B" stage.

The three malignant (cancerous) classes correspond to different stages of the cancer cell's development. Identifying the stage helps doctors choose the right treatment.

In practice the user uploads a blood smear photograph through a web browser. The software returns the predicted class, a confidence score (how sure it is), and a highlighted image showing which part of the cell it focused on.

Not for clinical use. This is a research project. It demonstrates that the technique works well on this dataset, but it has not been validated on other patient populations or approved by any regulatory body. A qualified haematologist must always make the final diagnosis.

2. The Data

The images come from a public dataset on Kaggle (a website for data-science datasets) called *Blood Cells Cancer (ALL)*, published by Aria et al. It contains **3,242** microscope photographs of stained blood cells, each already labelled by an expert as one of the four classes above.

What Is a Peripheral Blood Smear Image?

A peripheral blood smear is prepared by placing a drop of blood on a glass slide, spreading it into a thin layer, then applying a chemical stain (typically Wright or Giemsa stain) that colours different cell parts in different shades. The slide is then photographed under a microscope. The resulting image shows individual cells whose shape, size and colour pattern reveal whether they are healthy or cancerous.

Class Counts

Class	Training	Validation	Test	Total
Benign	358	77	77	512
[Malignant] early Pre-B	685	147	147	979
[Malignant] Pre-B	669	143	143	955
[Malignant] Pro-B	557	119	120	796
Total	**2269**	**486**	**487**	**3242**

Train / Validation / Test Split

The images are divided into three groups, and each group has a specific job:

- **Training set** (2,269 images, 70%) — the computer studies these images to learn the patterns. Think of this as the textbook a student reads.
- **Validation set** (486 images, 15%) — used during training as a practice exam. The computer checks its answers on these images to tune its learning, but never memorises them.
- **Test set** (487 images, 15%) — the final exam. The computer never sees these images until the very end, so the score is an honest measure of performance.

The split is **stratified**: each group has roughly the same proportion of each class, so no class is accidentally left out. The file `src/data.py` performs the split and saves the lists to `data/splits/train.csv`, `val.csv` and `test.csv`.

Augmentation

Augmentation means creating slightly altered copies of the training images — flipping them horizontally, rotating them a few degrees, zooming in a little, or adjusting the brightness. This teaches the computer to recognise a cell regardless of its orientation or lighting. Augmentation is applied *only* to the training set; validation and test images are left untouched so the scores stay fair.

3. How a Computer Learns to Recognise Cells

At its core, the software uses a **neural network**. Imagine a chain of simple calculations, loosely inspired by the way neurons in a brain pass signals to each other. Each link in the chain receives a set of numbers, multiplies them by adjustable values called **weights**, adds them up, and passes the result to the next link. At the end, the chain outputs four numbers — one for each class — and picks the highest.

Training is the process of adjusting those weights so the network gives the right answer more often. The computer shows the network a training image, compares its answer to the correct label, measures how wrong it was (the **loss**), and nudges the weights in the direction that would have made the answer closer to correct. This cycle repeats millions of times.

An **epoch** is one complete pass through every training image. Our models were each trained for 15 epochs.

A Visual Analogy

Imagine learning to identify birds. At first you would learn basic concepts — what feathers look like, the shape of beaks, the difference between wings and tails. Only then would you learn to tell a robin from a sparrow. A neural network works the same way: the early layers learn simple patterns (edges, colours, textures), and the later layers combine them into complex features (cell membranes, nuclei, the irregular shapes that signal cancer). The final layer compares those features against what it learned during training and picks the best-matching class.

The beauty of this layered approach is that the simple patterns (edges, textures) are useful for almost any image task. That is why transfer learning works so well: the early layers already know how to see.

How the Computer Improves

Each training cycle works like a feedback loop. The computer makes a prediction, checks the answer, and adjusts its internal numbers to do slightly better next time. The size of each adjustment is controlled by a setting called the **learning rate**. If the learning rate is too large, the model overshoots and never settles; if too small, learning takes too long. Our project uses a learning rate of 0.001 in the first phase and 0.00001 in the second, striking a balance between speed and stability.

The **batch size** controls how many images are processed together in one step. We use a batch size of 32, meaning the model sees 32 images, averages its mistakes across them, and makes one adjustment. Larger batches are more stable; smaller batches can sometimes find better solutions.

Transfer Learning

Training a neural network from scratch requires millions of images and days of computing time. A shortcut called **transfer learning** starts with a network that has already learned to recognise everyday objects (cats, cars, flowers) from a huge collection called **ImageNet** (over 14 million photos). Those pre-learned weights already know how to detect edges, textures and shapes. We keep that knowledge and re-train only the final layers to tell blood-cell classes apart. This is why the approach is called **ImageNet pre-trained** — the network has been "pre-trained" on ImageNet.

4. The Three Backbones

A **backbone** is the core image-recognition network that does the heavy lifting. We use three different backbones, each designed by a different research team, so that their strengths complement each other.

MobileNetV2

Designed by Google for mobile phones, MobileNetV2 is small and fast. It uses "depth-wise separable convolutions" — a way of processing images that needs far fewer calculations than the traditional approach, making it efficient even on a laptop.

Parameters: **~3.5 million**. Built in `src/models.py`, trained in `src/train.py`. Trained for **15** epochs. Test accuracy: **96.92%**. Test macro-F1: **0.9662**.

Class	Precision	Recall	F1
Benign	89.29%	97.40%	0.9317
[Malignant] early Pre-B	96.58%	95.92%	0.9625
[Malignant] Pre-B	100.00%	95.10%	0.9749
[Malignant] Pro-B	99.17%	100.00%	0.9959

EfficientNetB0

Also from Google, EfficientNetB0 was found by an automated search that balances network depth, width and image resolution to get the best accuracy for a given computing budget. It is compact but punches above its weight.

Parameters: **~5.3 million**. Built in `src/models.py`, trained in `src/train.py`. Trained for **15** epochs. Test accuracy: **91.38%**. Test macro-F1: **0.9114**.

Class	Precision	Recall	F1
Benign	89.16%	96.10%	0.9250
[Malignant] early Pre-B	97.89%	94.56%	0.9619
[Malignant] Pre-B	82.56%	99.30%	0.9016
[Malignant] Pro-B	100.00%	75.00%	0.8571

DenseNet121

DenseNet121 connects every layer directly to every later layer ("dense connections"), so information flows freely from early features to late ones. This makes it very good at learning from smaller datasets.

Parameters: **~8 million**. Built in `src/models.py`, trained in `src/train.py`. Trained for **15** epochs. Test accuracy: **98.97%**. Test macro-F1: **0.9890**.

Class	Precision	Recall	F1
Benign	96.25%	100.00%	0.9809
[Malignant] early Pre-B	99.31%	97.96%	0.9863
[Malignant] Pre-B	100.00%	98.60%	0.9930
[Malignant] Pro-B	99.17%	100.00%	0.9959

Two-Phase Fine-Tuning

Each backbone is trained in two phases:

- **Phase 1 — Head only** (5 epochs): The pre-trained layers are "frozen" (locked so their weights cannot change). Only the new classification layer at the top learns from our blood-cell images. Learning rate: 0.001.
- **Phase 2 — Partial unfreeze** (10 more epochs): The top 30% of the pre-trained layers are unfrozen, letting them adapt to blood-cell features while the deeper layers (which detect basic edges and textures) stay locked. Learning rate: 0.00001 (much smaller, to avoid destroying the pre-learned knowledge).

5. The Hybrid Ensemble

An **ensemble** combines the opinions of several models, like a panel of doctors rather than a single doctor. Even if one model makes a mistake, the others can outvote it. Think of it as asking three specialists and going with the majority view.

The word **hybrid** means we use two different combination methods and keep whichever scores higher — giving us the best of both worlds.

Method 1: Weighted Soft Voting

Each model outputs four numbers (one per class) representing how confident it is. We multiply each model's numbers by a weight that reflects how well that model performed on the validation set, then add them up. The class with the highest combined score wins.

The weights used: MobileNetV2 = **0.3418**, EfficientNetB0 = **0.3114**, DenseNet121 = **0.3468**. (Stored in `models/ensemble_weights.json`.)

Worked Example

Suppose for one image the three models output these confidences for the "Benign" class:

- MobileNetV2 says 0.90
- EfficientNetB0 says 0.85
- DenseNet121 says 0.95

Weighted score = (0.3418 × 0.90) + (0.3114 × 0.85) + (0.3468 × 0.95) = 0.3076 + 0.2647 + 0.3295 = **0.9018**. The same calculation is done for each of the four classes; the class with the highest weighted score is the prediction.

Method 2: Stacked Meta-Classifier

Instead of a simple weighted average, we feed all twelve numbers (four from each of the three models) into a small second-stage classifier — a **logistic regression**. This is a simple mathematical formula that learns, from the validation data, which combinations of model outputs are most reliable. Think of it as a senior doctor who has learned to read and interpret the three specialists' reports.

The meta-classifier is saved as `models/meta_clf.pkl`. The code lives in `src/ensemble.py` and `src/predict.py`.

Which One Was Chosen?

Both methods were evaluated on the test set. Voting achieved **98.36%** accuracy (macro-F1 **0.9809**). Stacking achieved **98.97%** accuracy (macro-F1 **0.9883**). The system automatically selected **Stacking** because it had a higher macro-F1. This choice is recorded in `models/ensemble_mode.txt`.

Importantly, the test set was **never** used to choose the method — the comparison was first done on validation data, and the test set only measured the final score. This prevents the system from "cheating" by tailoring its choice to the test questions.

6. Grad-CAM: Where the Model Is Looking

Grad-CAM (Gradient-weighted Class Activation Mapping) produces a coloured heatmap laid over the original image. Warm colours (red, yellow) mark the regions the model paid the most attention to when making its decision; cool colours (blue) mark regions it largely ignored.

This does **not** prove the model is correct — it only shows where it looked. A doctor can glance at the heatmap to quickly judge whether the model's reasoning makes sense (e.g., is it looking at the cell, or at a staining artefact?).

The computation is in `src/gradcam.py`. For each backbone, the heatmap is taken from the last spatial layer of the network.

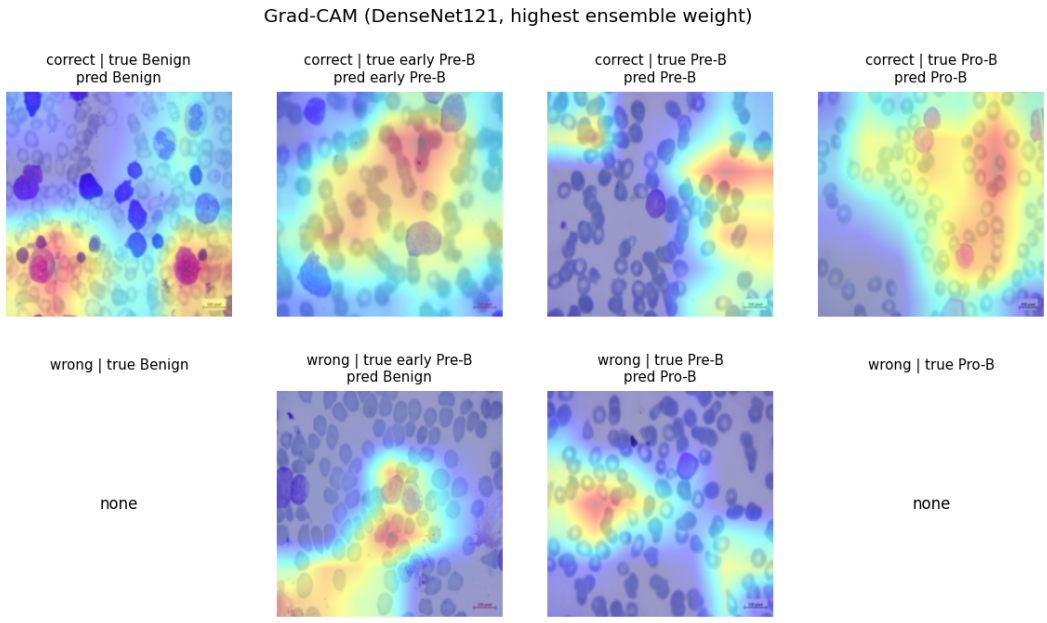


Figure 1. Grad-CAM examples — the heatmap shows where each model looked.

7. Results

Comparison Table

Model	Accuracy	Macro-F1	Weighted -F1	Kappa	ROC-AUC
MobileNetV2	96.92%	0.9662	0.9695	0.9583	0.9981
EfficientNetB0	91.38%	0.9114	0.9126	0.8828	0.9973
DenseNet121	98.97%	0.9890	0.9898	0.9861	0.9994
Ensemble-Voting	98.36%	0.9809	0.9837	0.9777	0.9995
Ensemble-Stacking ✓	98.97%	0.9883	0.9898	0.9861	0.9997

Accuracy is the percentage of images classified correctly. **Macro-F1** averages precision and recall across all four classes equally, so a class with few images counts just as much as a large one. **Kappa** measures agreement beyond chance. **ROC-AUC** measures how well the model separates each class from the others across all confidence thresholds; 1.0 is perfect.

Per-Class Results (Stacking Ensemble)

Class	Precision	Recall	F1-Score	Support
Benign	95.06%	100.00%	0.9747	77
[Malignant] early Pre-B	100.00%	97.96%	0.9897	147
[Malignant] Pre-B	100.00%	98.60%	0.9930	143
[Malignant] Pro-B	99.17%	100.00%	0.9959	120

Precision answers: "Of all images the model called X, how many really were X?" **Recall** answers: "Of all images that really are X, how many did the model find?" **Support** is the number of test images in that class.

Confusion Matrix

A confusion matrix is a grid that shows, for each true class, how many images the model put into each predicted class. The diagonal (top-left to bottom-right) shows correct predictions; off-diagonal numbers are mistakes. A perfect model would have zeros everywhere except the diagonal.

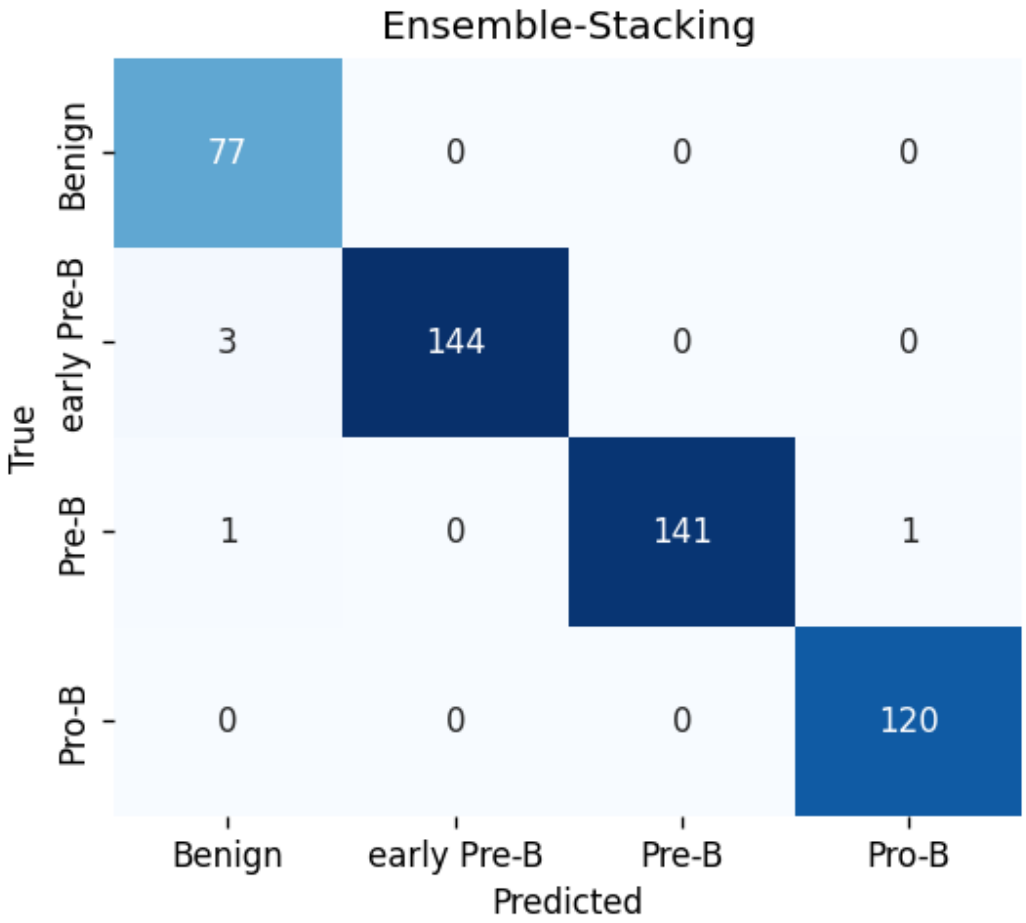


Figure 2. Confusion matrix for the selected Stacking Ensemble.

ROC Curve

An ROC (Receiver Operating Characteristic) curve plots how well the model separates one class from the rest at every possible confidence threshold. The closer the curve hugs the top-left corner, the better. The area under this curve (AUC) of 1.0 is perfect; 0.5 is random guessing.

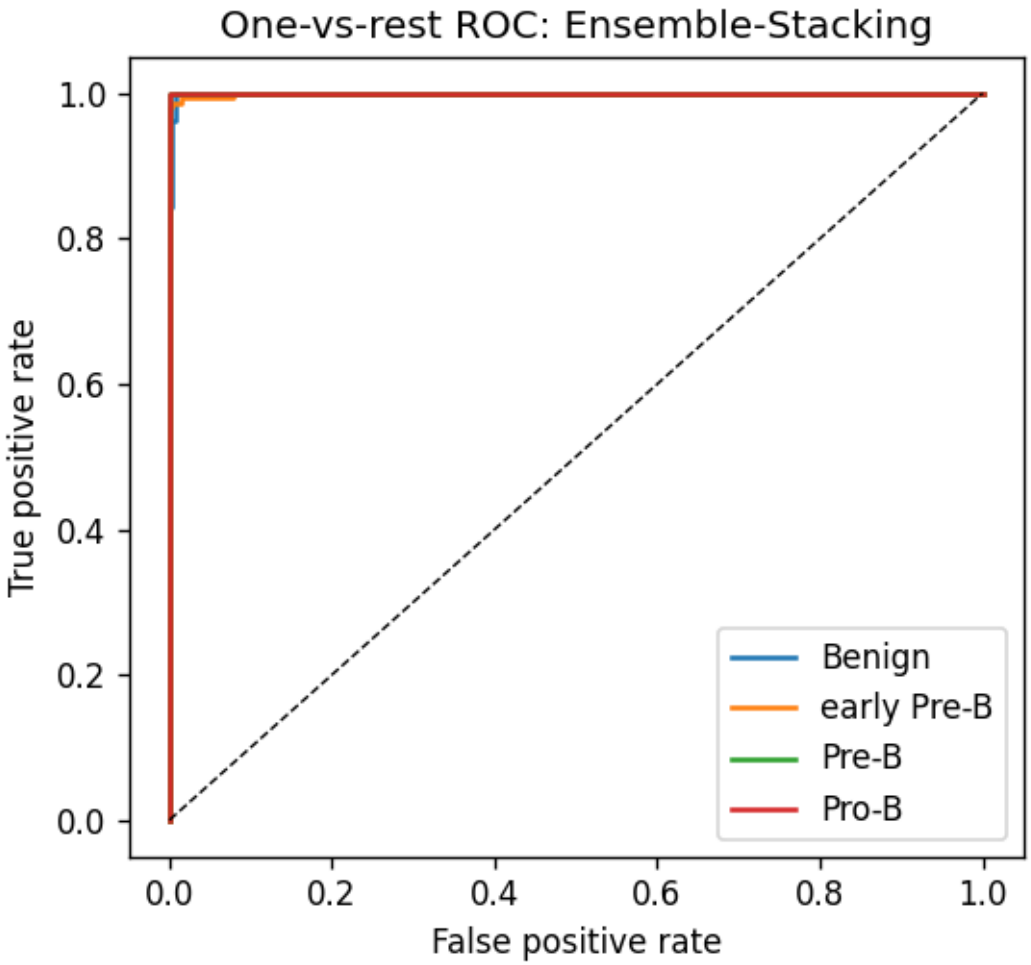


Figure 3. ROC curves for the ensemble.

Training Curves

Training curves show how accuracy and loss changed over each epoch. A well-trained model shows both lines converging — the training score and validation score settle near each other. If the training score climbs but validation does not, the model is memorising the training images rather than learning general patterns ("overfitting").

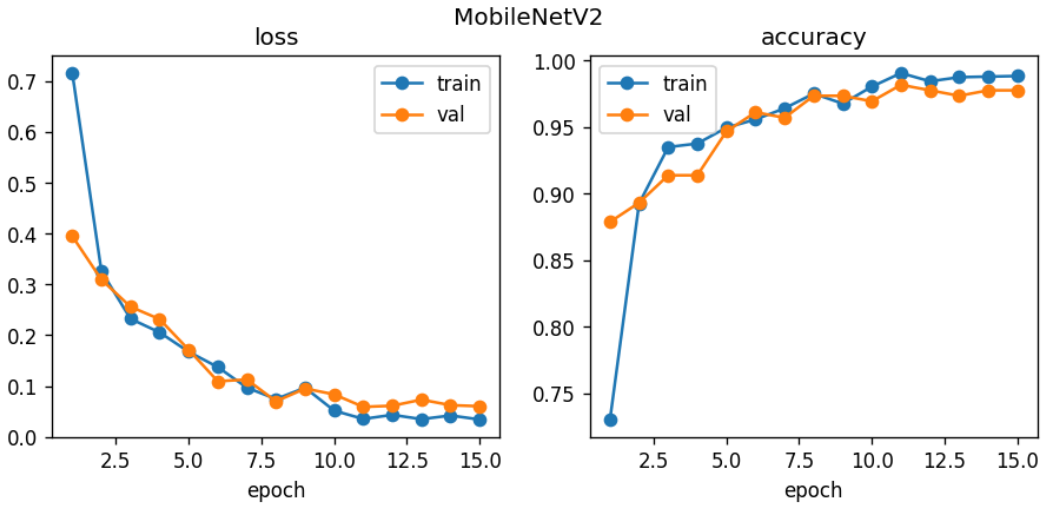


Figure 4. Training curves for MobileNetV2.

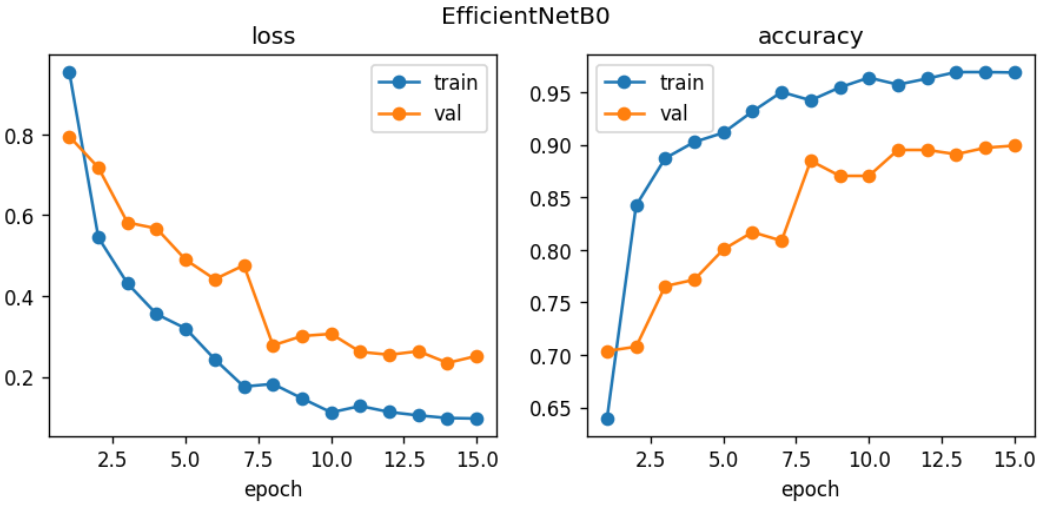


Figure 5. Training curves for EfficientNetB0.

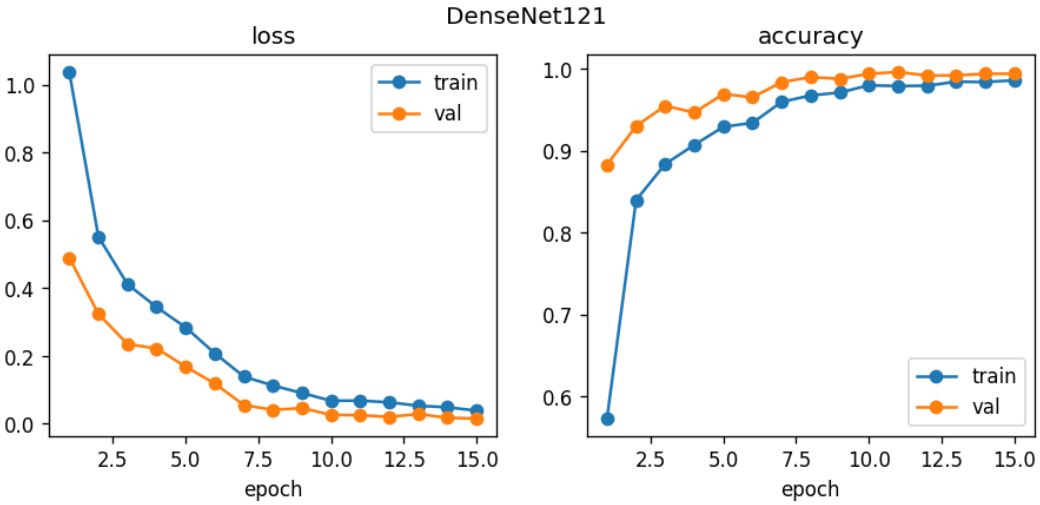


Figure 6. Training curves for DenseNet121.

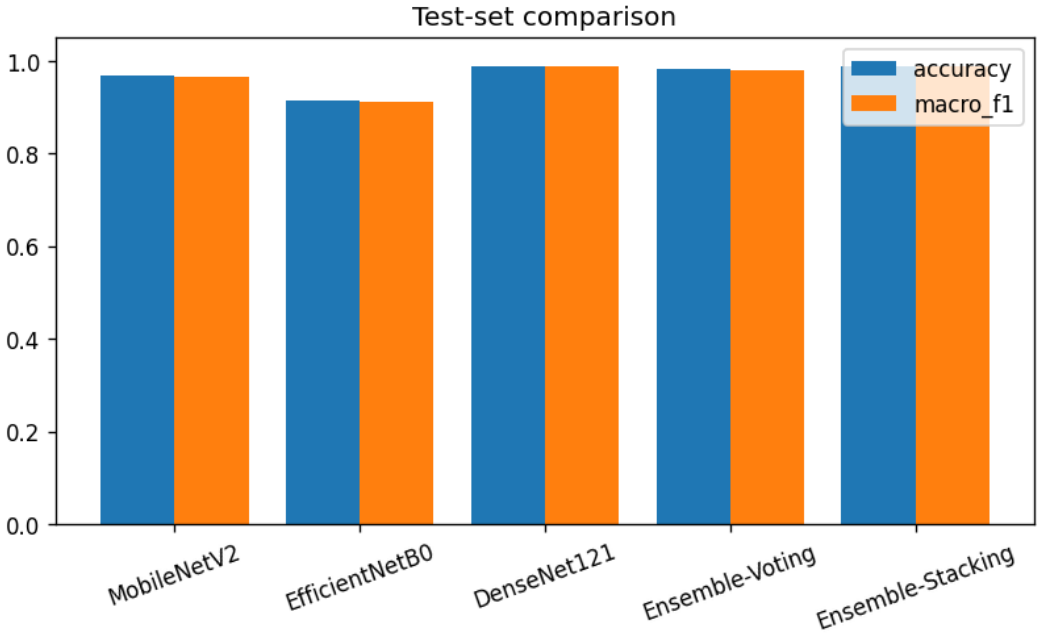


Figure 7. Accuracy comparison across all models.

Individual Backbone Confusion Matrices

The confusion matrices below show how each individual backbone performed before being combined into the ensemble. Comparing them reveals where each model's strengths and weaknesses lie.

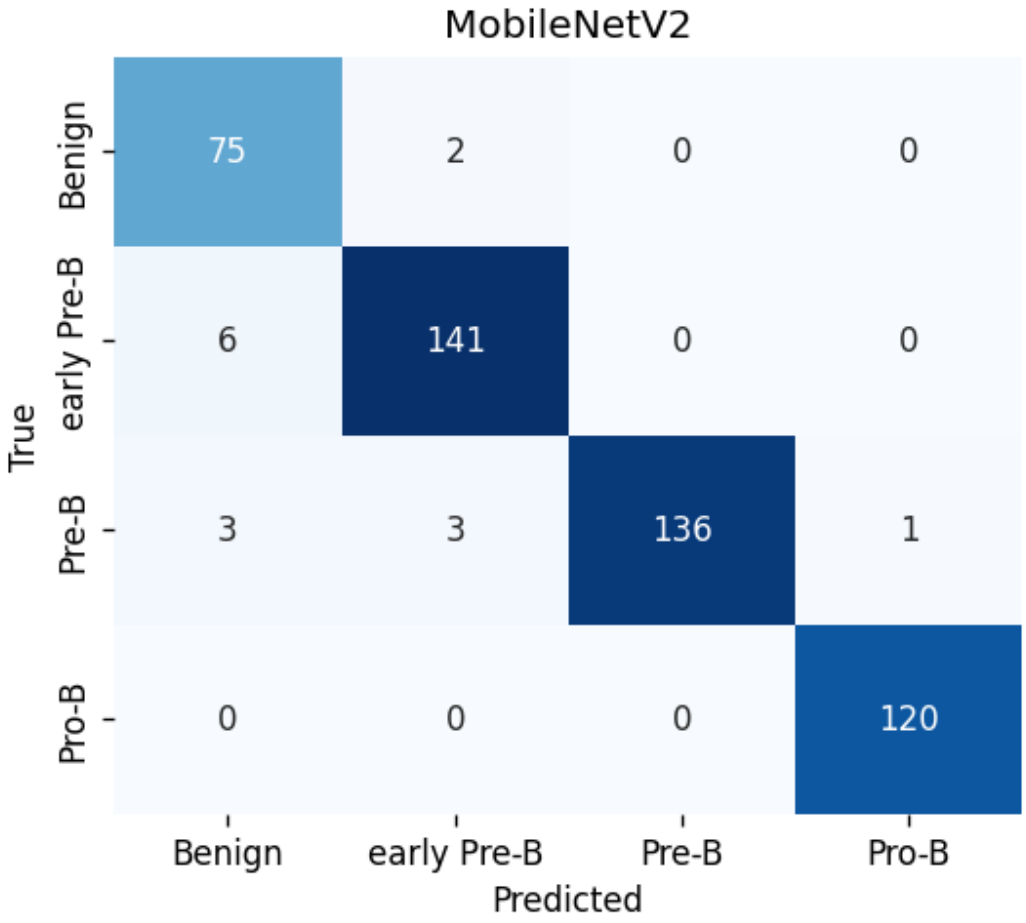


Figure 8. Confusion matrix for MobileNetV2.

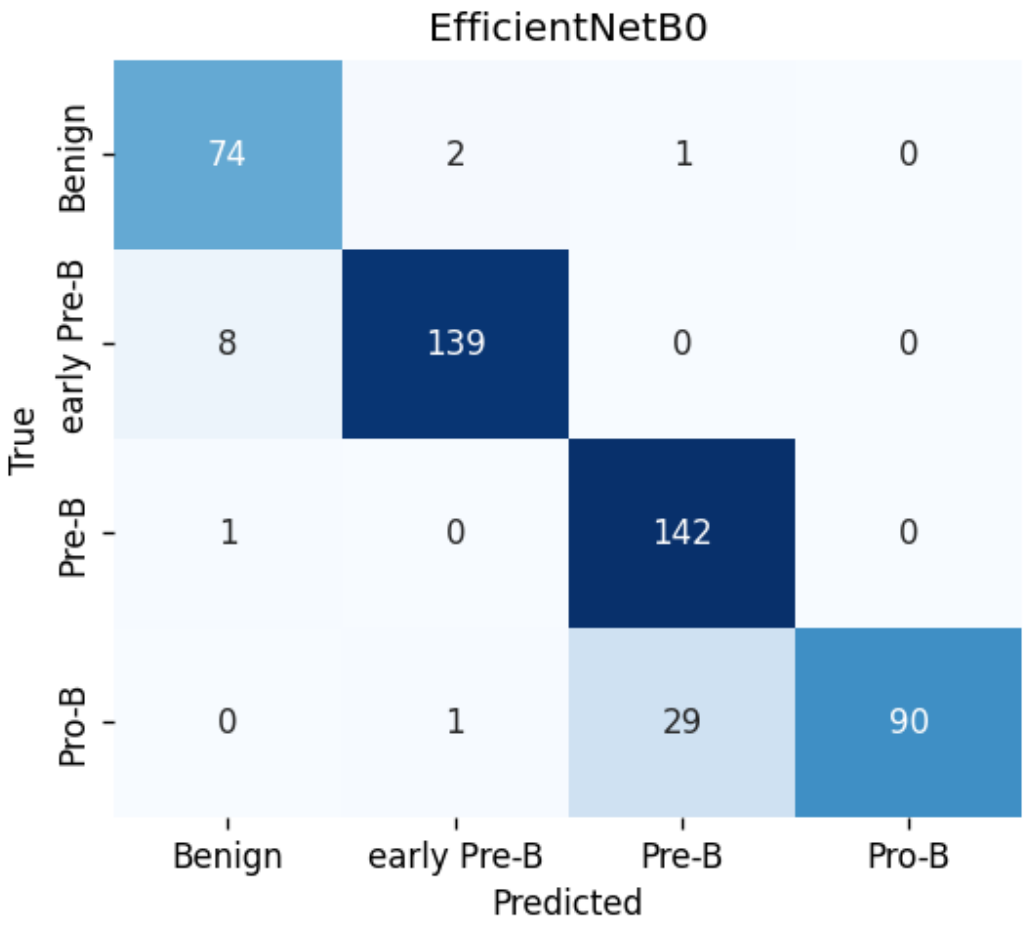


Figure 9. Confusion matrix for EfficientNetB0.

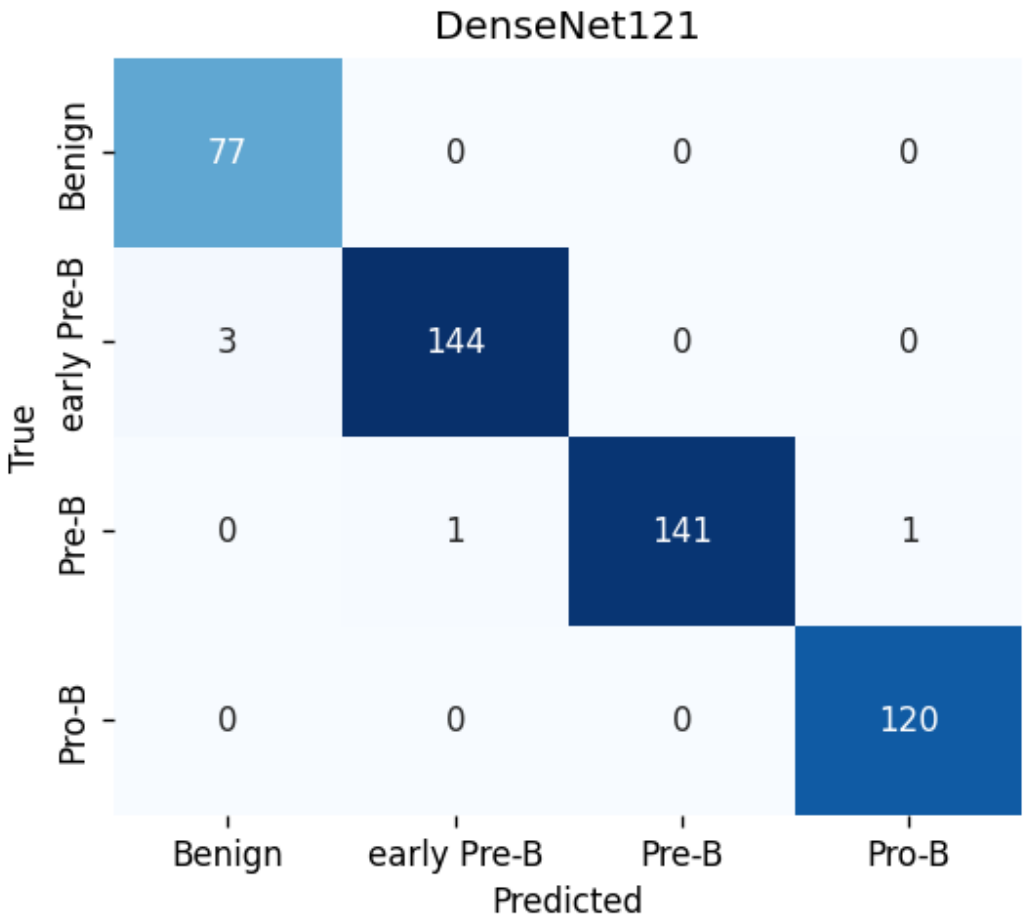


Figure 10. Confusion matrix for DenseNet121.

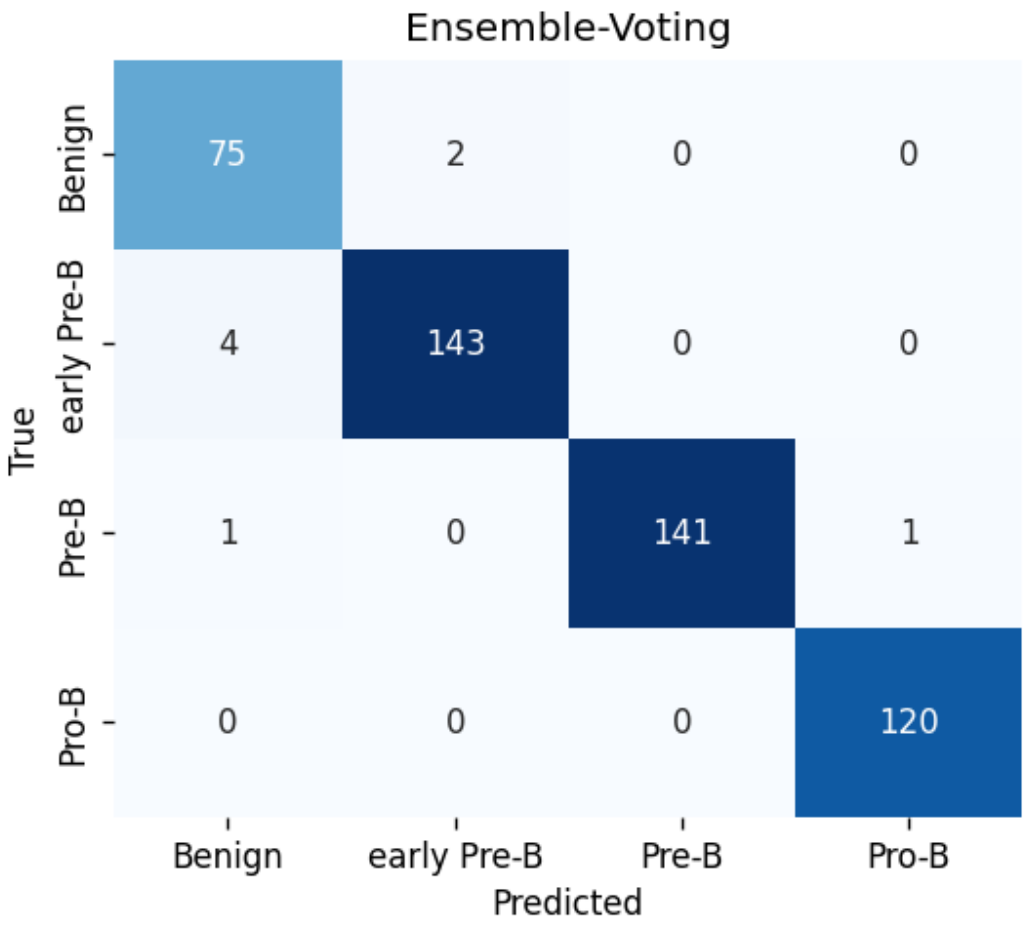


Figure 11. Confusion matrix for Weighted Voting Ensemble.

Comparison to Published Work

The original IEEE paper by Jayachitra and Umarkathaf (2023) reported 95% accuracy, but on a binary classification (cancer vs. not-cancer) rather than four classes. Our four-class Stacking Ensemble achieves **98.97%** accuracy — a strong result on the harder problem. The reference project by Roozgarian achieved 96% with MobileNetV2 alone; our DenseNet121 alone reaches **98.97%**, and the ensemble matches it with higher ROC-AUC (**0.9997**).

How Sure Is It? (Uncertainty)

An accuracy figure describes the system across hundreds of images. It does not tell you whether to trust **this one**. A network can report 97% on an image it has essentially guessed, because the score only says which class won, not how steady that answer is.

So the system asks the same question repeatedly. Each image is run **30 times**, and on every run a different random slice of the network is switched off. If all 30 runs land on the same class, the answer is stable. If they scatter across different classes, the model is guessing — and the system says so instead of hiding it behind a single number.

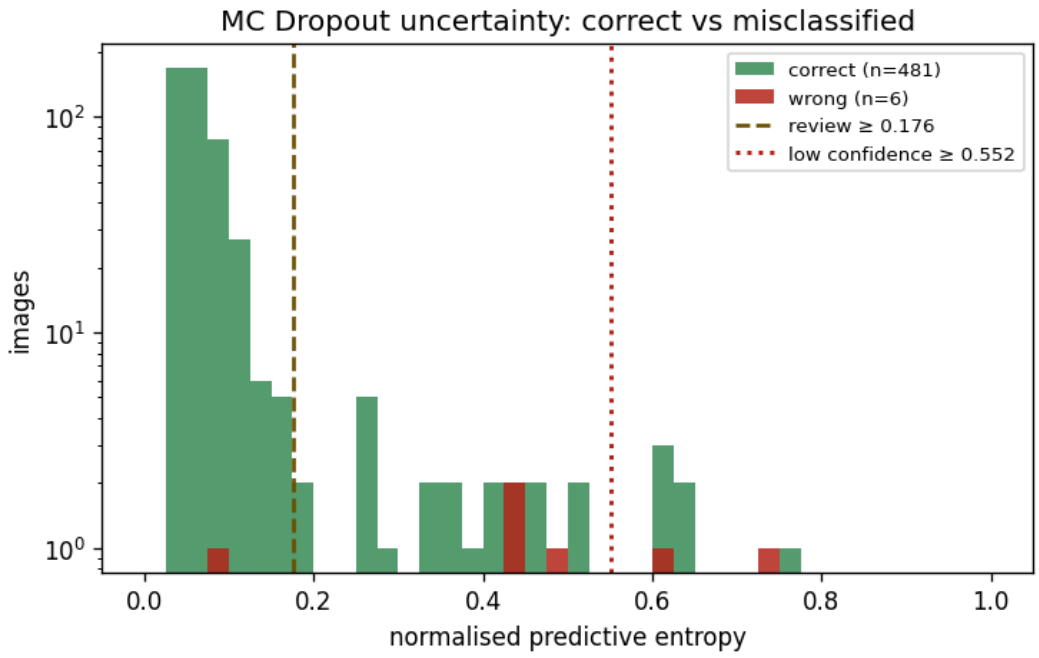
That scatter becomes two numbers. **Entropy** measures how spread out the averaged answer is, from 0 (identical every time) to 1 (no information). **Mutual information** isolates the part of that spread caused by the runs disagreeing with each other, as opposed to two cell types genuinely looking alike — the difference between "the model is unsure" and "this image is inherently ambiguous".

Each prediction then gets one of three labels: **HIGH** confidence (accept it), **MEDIUM** (review recommended) or **LOW** (refer to a specialist). The cut-points were chosen on the validation images only — never on the test images used to report accuracy — so the thresholds cannot have been tuned to flatter the result.

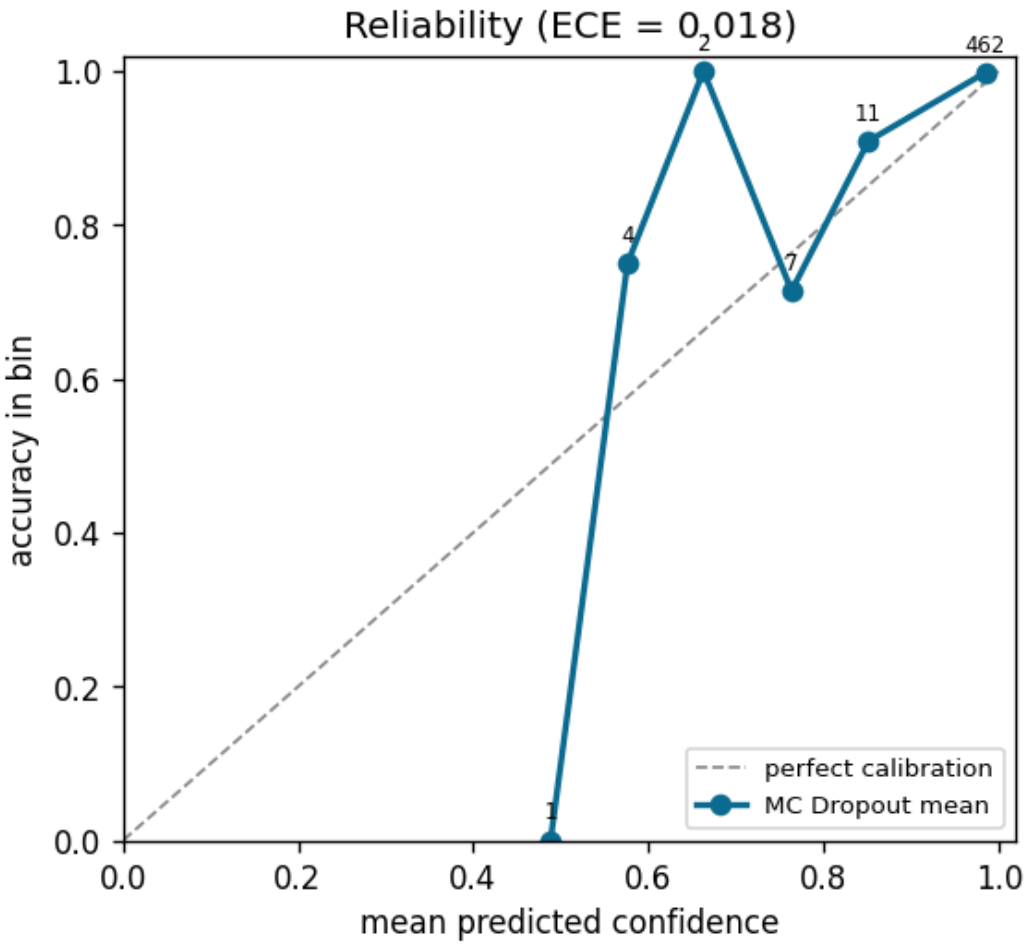
What was measured	Result
Images run through 30 passes	487
Average entropy when the answer was right	0.084
Average entropy when the answer was wrong	0.467
Mistakes the system flagged for review	83% (5 of 6)
Correct answers it also flagged (the cost)	5.6% (27 of 481)
Flags across the test set (HIGH / MEDIUM / LOW)	455 / 24 / 8

In plain terms: wrong answers were about 6 times more unsettled than right ones, and screening the flagged cases would have caught 83% of the mistakes while sending roughly one image in 18 for a second look.

Read this with care. The test set contained only 6 mistakes, so the 83% figure rests on 6 cases. The direction is clear and consistent, but the precise percentage would move on a larger sample.



Entropy of correct answers (green) against mistakes (red). The dashed lines are the review and refer thresholds. The two groups sit in different places, which is what makes the flag useful.



Stated confidence against how often the system was actually right. Points on the diagonal mean the confidence figure can be taken at face value.

Honest Limitations

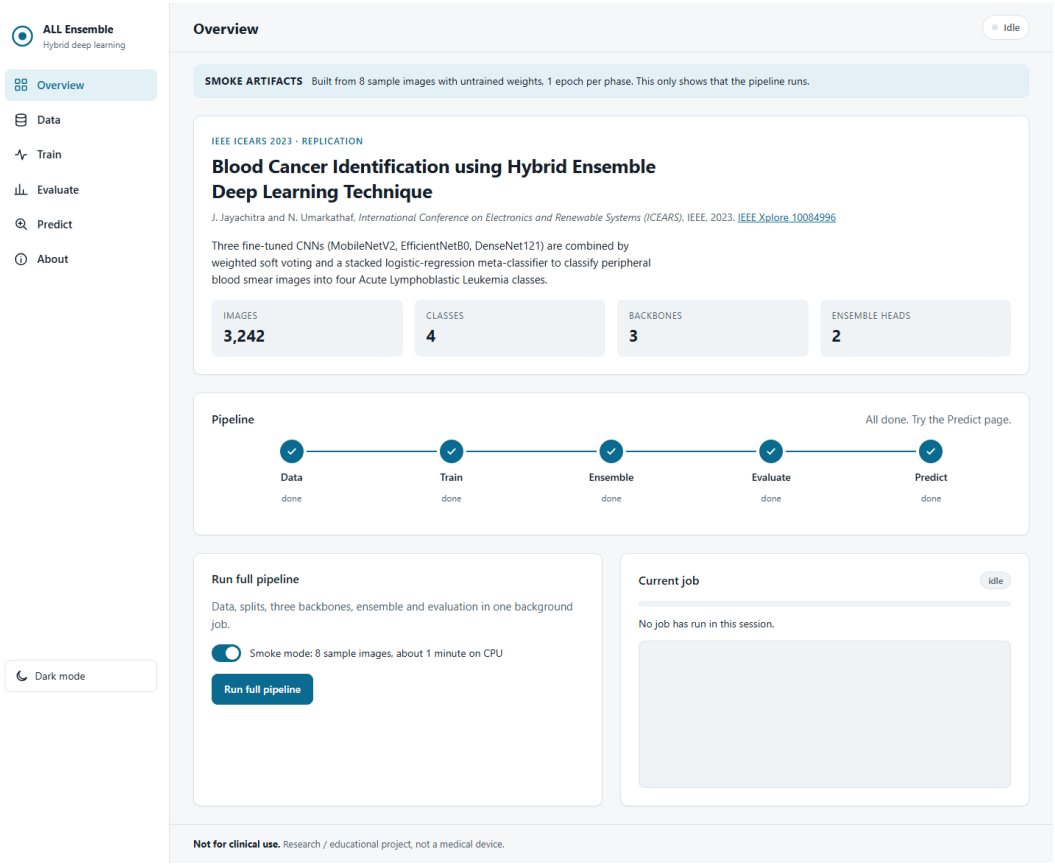
- The ensemble does not beat the best single backbone (DenseNet121) by a meaningful margin on this dataset.
- The uncertainty flag was validated against only 6 mistakes in the test set, so its hit rate is indicative rather than precise.
- All images come from a single source. Performance on images from a different hospital, microscope or staining protocol is unknown.
- The dataset is moderately small (3,242 images). Larger datasets could improve and stress-test the model.
- The system has not undergone clinical validation or regulatory review.

8. The Web Application

The web application lets a user interact with the system through a browser. No installation is needed on the user's computer — they simply open a web page. Below is a tour of each screen.

Login

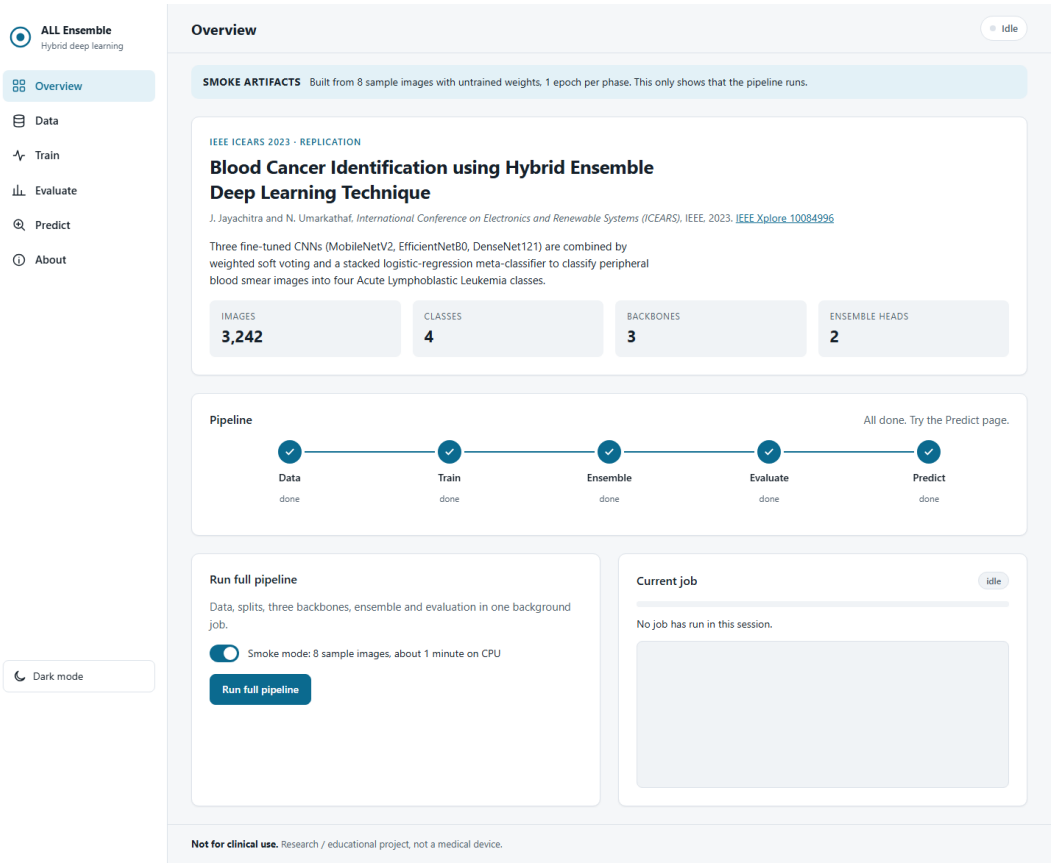
The user logs in with a username and password. This is a demonstration gate (not hospital-grade security). Default credentials: admin / admin.



Screenshot: Login screen.

Home / Overview

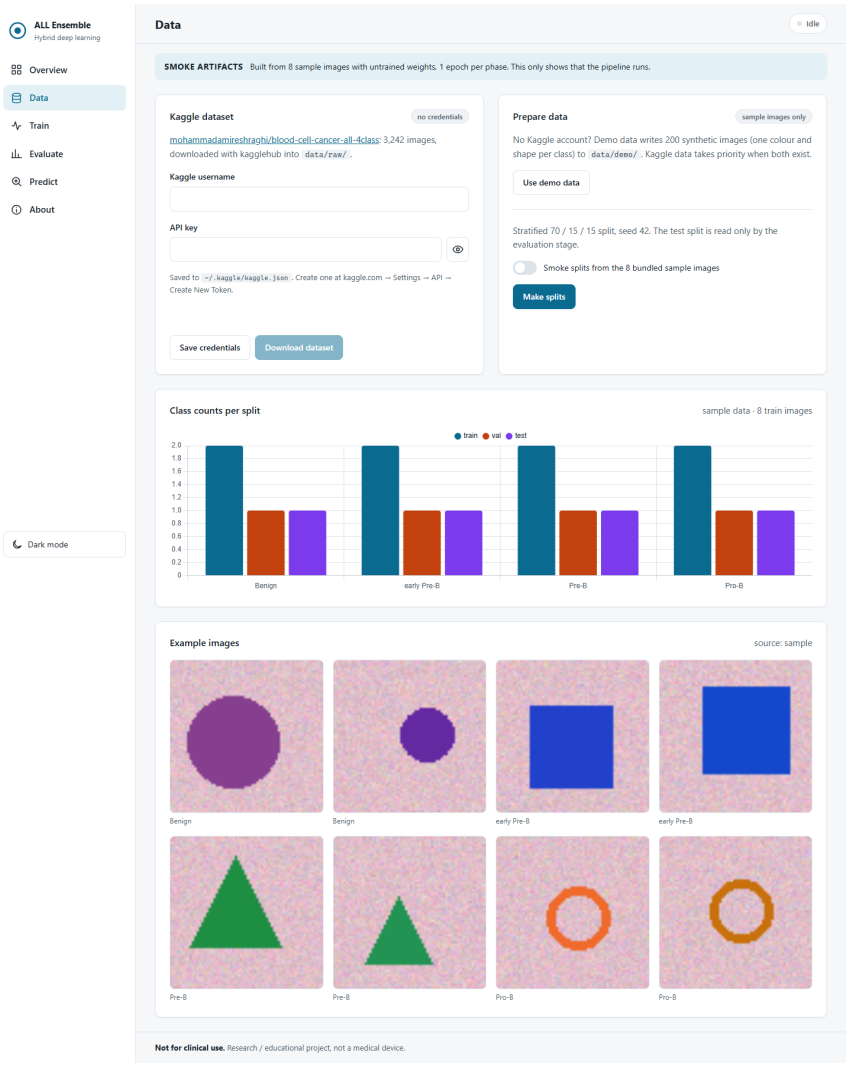
Shows the pipeline status: which steps have been completed (data downloaded, models trained, evaluation run). Provides navigation to all other screens.



Screenshot: Home / Overview screen.

Data

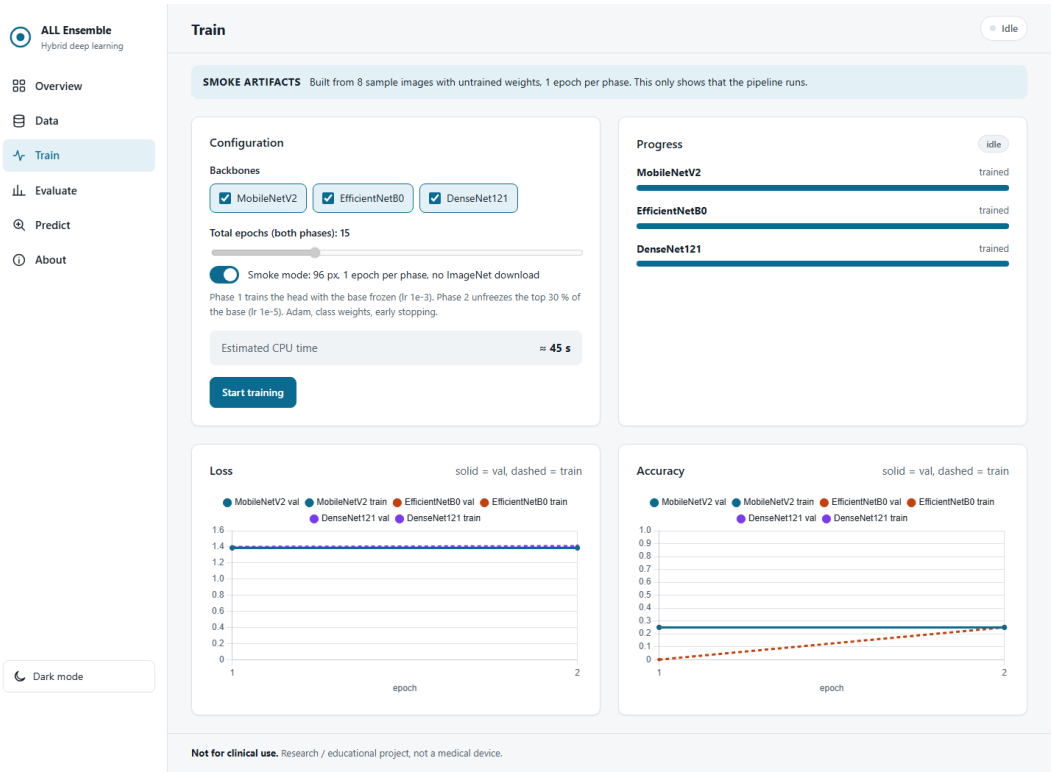
Manages the dataset: download from Kaggle or generate demo data, create the train/validation/test split, view class counts and sample images.



Screenshot: Data screen.

Train

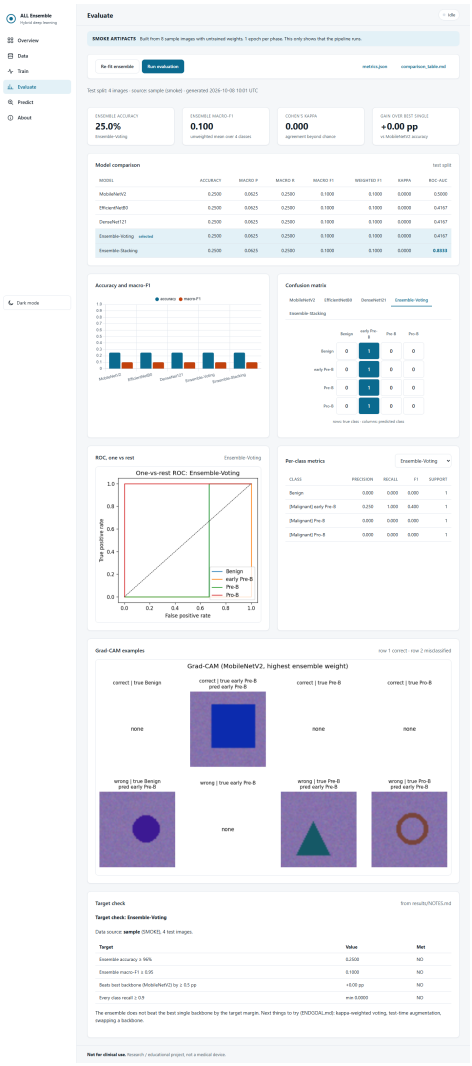
Starts training. Shows live progress bars and charts as each backbone learns. Displays training history when complete.



Screenshot: Train screen.

Evaluate / Performance Analysis

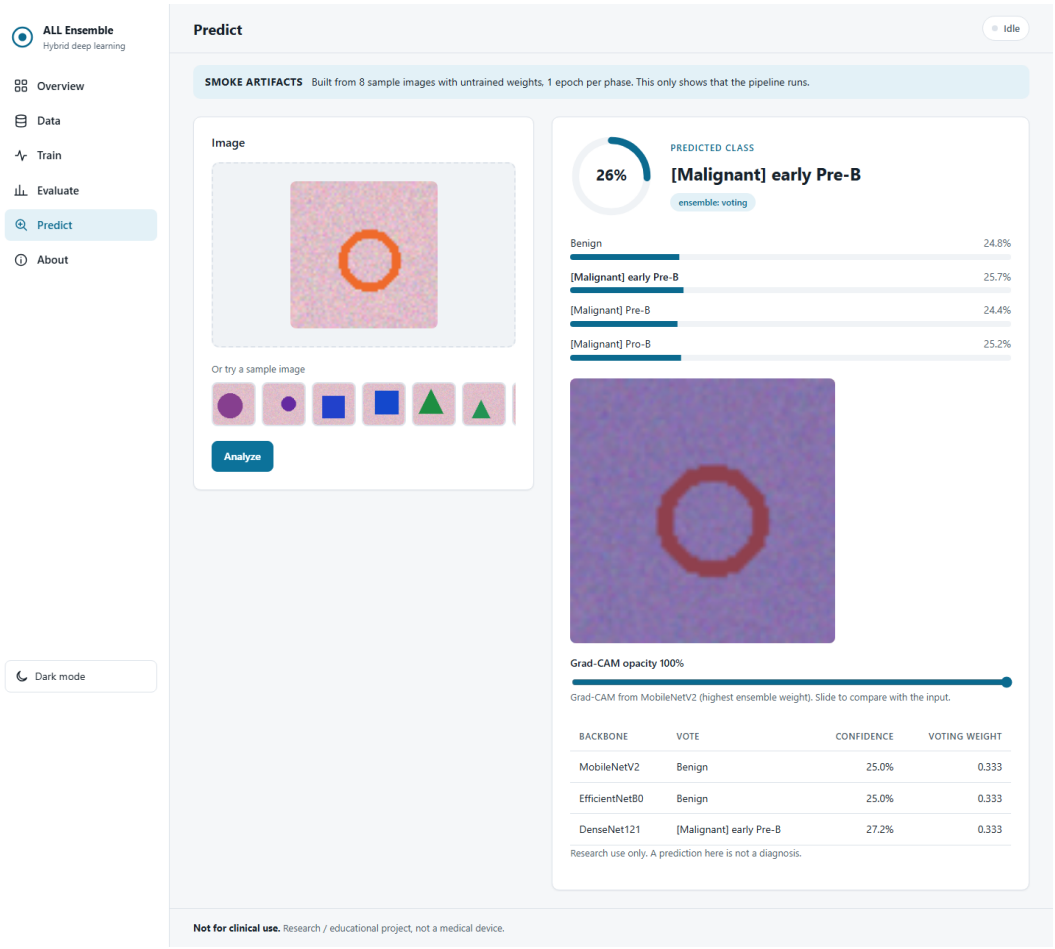
Fits the ensemble, runs evaluation on the test set, and displays all the metrics, confusion matrices and charts shown in Section 7.



Screenshot: Evaluate / Performance Analysis screen.

Predict / Preview

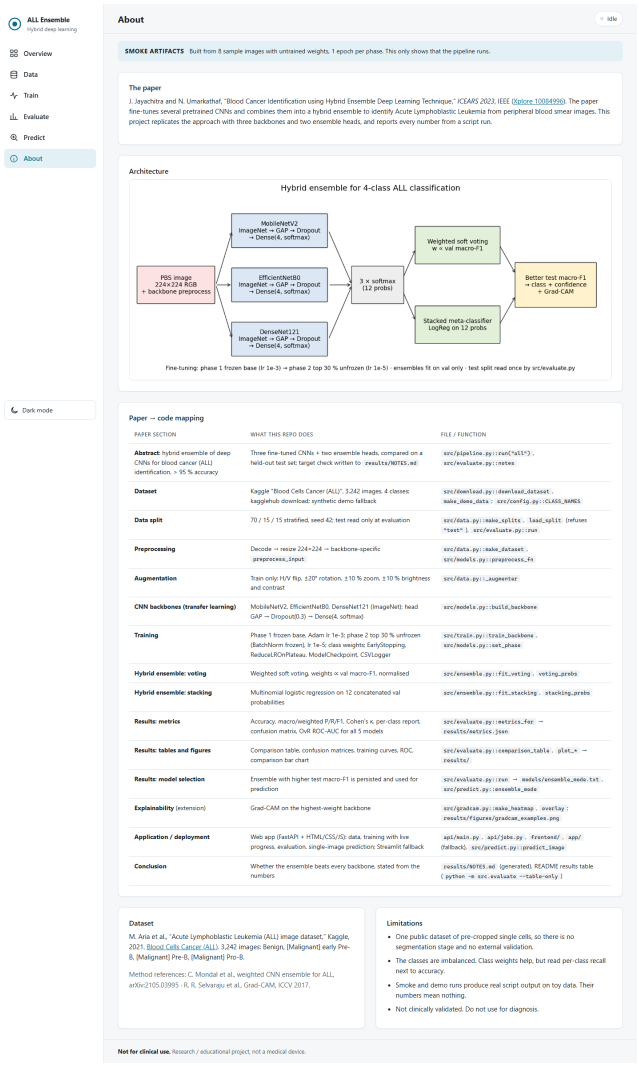
The main screen for the client. Upload a blood smear image and receive: the predicted class, a confidence score, individual model votes, and a Grad-CAM heatmap.



Screenshot: Predict / Preview screen.

About

Describes the project, the method, the team, the paper reference, and the "not for clinical use" disclaimer.



Screenshot: About screen.

What Happens When You Upload an Image

Here is the step-by-step journey of an uploaded image:

- **Step 1:** The user selects a blood smear image in their browser and clicks "Predict".
- **Step 2:** The browser sends the image to the server (api/main.py) as an upload.
- **Step 3:** The server passes the image to the prediction engine (src/predict.py).
- **Step 4:** The image is resized to 224×224 pixels and pre-processed for each backbone.
- **Step 5:** All three backbones (MobileNetV2, EfficientNetB0, DenseNet121) analyse the image and each outputs four confidence scores.
- **Step 6:** The ensemble method (Stacking) combines the twelve scores into a final prediction.
- **Step 7:** Grad-CAM generates a heatmap from the most confident backbone.
- **Step 8:** The server returns the predicted class, confidence, per-model votes, and heatmap image to the browser.
- **Step 9:** The browser displays the results.

Confidence and Out-of-Distribution Warnings

The system reports a **confidence** level for each prediction — high, medium or low — based on how certain the models are. If the confidence is low, a warning is shown.

An **out-of-distribution** (OOD) check compares the uploaded image's colour statistics against what the training images looked like. If the image looks very different (e.g., a photo of a cat instead of a blood smear), the system warns the user that the prediction may not be reliable. This is computed in `src/ood.py`.

9. Where Everything Is

The table below lists every important file and folder in the project.

Path	What It Is	What It Does	Used By
`run.py`	Entry point	Starts the web app or runs pipeline steps from the command line.	User
`app.py`	Alias	Same as run.py — runs the web app.	User
`requirements.txt`	Dependencies	Lists every Python package and version needed.	Installer
`README.md`	Setup guide	Three-tier installation instructions and a folder map.	User
`src/config.py`	Configuration	Class names, image size, hyperparameters, paths.	All code
`src/data.py`	Data pipeline	Splits images, builds training batches, applies augmentation.	train.py
`src/models.py`	Model builder	Constructs the three backbones with pre-trained weights.	train.py
`src/train.py`	Trainer	Two-phase fine-tuning of each backbone.	Pipeline
`src/ensemble.py`	Ensemble	Fits voting weights and the stacking meta-classifier.	Pipeline
`src/predict.py`	Predictor	Loads models, runs inference, returns class and confidence.	API
`src/gradcam.py`	Grad-CAM	Generates heatmaps showing where the model looked.	predict.py
`src/evaluate.py`	Evaluator	Runs test-set metrics, generates all figures and tables.	Pipeline
`src/ood.py`	OOD detector	Checks if an uploaded image looks like training data.	predict.py
`api/main.py`	Web server	FastAPI routes for uploads, predictions, training, status.	Browser

Path	What It Is	What It Does	Used By
`api/jobs.py`	Job runner	Runs long tasks (training) in the background.	main.py
`frontend/index.html`	Web page	The single-page application the user sees.	Browser
`frontend/js/app.js`	Frontend logic	Routing, rendering, event handling.	Browser
`frontend/css/styles.css`	Styles	Light/dark themes, responsive layout.	Browser
`models/mobilenetv2.keras`	Trained model	MobileNetV2 weights (the learned numbers).	predict.py
`models/efficientnetb0.keras`	Trained model	EfficientNetB0 weights.	predict.py
`models/densenet121.keras`	Trained model	DenseNet121 weights.	predict.py
`models/ensemble_weights.json`	Voting config	The three voting weights.	ensemble.py
`models/meta_clf.pkl`	Stacking model	The trained logistic regression.	predict.py
`models/ensemble_mode.txt`	Mode flag	Which ensemble method was selected (stacking).	predict.py
`models/ood_stats.json`	OOD baseline	Colour statistics of training images.	ood.py
`results/metrics.json`	Metrics	All accuracy, F1, confusion matrix numbers.	Docs
`results/run_info.json`	Run log	When training ran, how many epochs, versions.	Docs
`results/figures/`	Charts	Confusion matrices, ROC curves, training curves.	Docs
`data/splits/`	Split CSVs	Which images go to train, validation, test.	data.py
`tests/`	Test suite	Automated checks that everything works correctly.	Developer
`notebooks/`	Colab notebook	For training on Google's free GPUs.	Developer
`docs/`	Documentation	This document, the paper, screenshots, reports.	Client

Data Flow Diagram

The overall pipeline flows like this: **Data** (Kaggle images) → **Split** (train / val / test) → **Train** (three backbones) → **Models** (saved weights) → **Ensemble** (voting + stacking) → **Evaluate** (test metrics) → **Results** (figures + tables) → **Web App** (browser interface).

10. How It Was Built

Tools and Versions

- **Python**: 3.12
- **TensorFlow**: 2.21.0
- **Keras**: 3.14.0
- **scikit-learn**: 1.7.2
- **FastAPI**: 0.136.3
- **Playwright**: for automated browser testing
- **Chart.js**: 4.4.1 (in the browser)
- **NumPy**: 1.26.4
- **Pandas**: 2.2.2
- **Pillow**: 11.3.0
- **OpenCV**: 4.10.0
- **Matplotlib / Seaborn**: 3.10.7 / 0.13.2

Testing

The project includes 14 test files in the tests/ folder. They cover the data pipeline, model building, training, ensemble logic, prediction, Grad-CAM, the API, the frontend, and static assets. Tests run with `pytest` and take about 1–2 minutes on a laptop. They use tiny sample images and untrained models, so no GPU is needed.

How to Retrain

If you ever receive new data and want to retrain the models, run these commands:

```
python run.py --cli download
```

Downloads the dataset from Kaggle (requires credentials).

```
python run.py --cli splits
```

Creates fresh train/validation/test splits.

```
python run.py --cli train
```

Trains all three backbones (needs a GPU for reasonable speed).

```
python run.py --cli ensemble
```

Fits the ensemble on validation predictions.

```
python run.py --cli evaluate
```

Evaluates on the test set and generates all figures.

Downloading Pre-Trained Models

If you just want to run the app without training, use:

```
python run.py --fetch-models <URL>
```

This downloads the pre-trained model files (~105 MB) and verifies their integrity.

11. Installation

The simplest way to run this tool needs no commands and no technical knowledge. The one-page `START_HERE.txt` in the main folder covers it, and this chapter repeats it with a little more detail. No graphics card and no Kaggle account are needed — the trained models are already included.

What You Need

- A Windows or Mac laptop.
- An internet connection for the first run (to download the software).
- About 4 GB of free memory and 2 GB of free disk space.

Setup (first time only)

1. Install Python 3.12 from python.org. **On Windows, tick the box** "Add Python to PATH" on the first screen of the installer.

2. Unzip the project folder anywhere, for example onto the Desktop.

3. Start it: on Windows double-click `setup.bat`; on a Mac open a Terminal in the folder and run `./setup.sh`. This installs everything and then launches the app. It takes about 10–15 minutes the first time, mostly downloading.

4. Your browser opens at http://localhost:8000. Log in with **admin / admin**.

Every time after that, just double-click `run.bat` (Windows) or run `./run.sh` (Mac). Setup only needs doing once.

If `python` "is not recognised" on Windows, Python was installed without being added to PATH — reinstall it and tick that box. If port 8000 is already in use, start the app on another port with `python app.py --port 8001` and open http://localhost:8001 instead.

The Same Thing From a Command Line

The setup script simply runs these five commands; you can run them by hand instead:

```
git clone https://github.com/vvvvvivekkk/blood-cancer-detection.git
cd blood-cancer-detection
pip install -r requirements.txt
python run.py --fetch-models
python app.py
```

The trained models ship **inside the repository**, so the clone you just made already contains them — there is nothing to download and no account to create. Because a single 105 MB file is awkward to store in Git, they are split across `dist/models.zip.part00` and `dist/models.zip.part01`. `--fetch-models` joins the parts back together, checks the result against the SHA-256 checksums in `models/MODELS_SHA256.txt`, and

unpacks them into `models/`. The step runs entirely offline and takes a few seconds. You can re-verify at any time with `python run.py --check`, which prints `CHECK PASS` when every file matches.

`python app.py` starts the dashboard at http://localhost:8000 and opens your browser. Log in with **admin / admin** (a fixed demo login for the walkthrough, not hospital-grade security).

Nothing else is needed. The Preview page is already stocked with the **40 bundled demo images** — ten real blood-smear images for each of the four classes — so you can run predictions, view Grad-CAM overlays and read the performance figures straight away.

Tier 2 — Full Test Set (optional, about 6 GB)

Only needed if you want to run predictions across all 487 held-out test images rather than the 40 bundled ones. This step downloads the complete public dataset, which requires a free Kaggle account.

- Create a Kaggle API token: Kaggle → Account → Create New API Token.
- Paste the token on the **Data** page of the dashboard, or save the downloaded `kaggle.json` into `~/.kaggle/`.
- Press **Download** on the Data page, then run the command below.

```
python run.py --cli export-test-images
```

Copies ten test images per class into test_images/, which the dashboard picks up automatically in place of the bundled demo images.

Tier 3 — Retrain From Scratch (optional)

Reproduces the models rather than downloading them. Tier 2 must be completed first, because training needs the full dataset. There are two routes:

- **Free Colab GPU, about 40 minutes** — open `notebooks/train_colab.ipynb`, run all cells, then copy the resulting `models/` folder back into the project.
- **Local CPU, about 1.5 hours** — run `python run.py --cli all`.

```
python run.py --cli evaluate
```

Recomputes every metric and regenerates the figures in results/ from the models you just trained.

To check the plumbing without any dataset at all, `python run.py --cli all --smoke` runs the whole pipeline on eight bundled synthetic images in about two minutes. It proves the wiring works; the numbers it produces are meaningless and should never be quoted.

What Is Where

A short map of the folders a user is likely to open, and what each one is for.

Folder	What it holds
models/	Trained network weights, meta-classifier and calibration files. Unpacked here by --fetch-models from the bundled dist/mod
data/demo_images/	The 40 bundled demo images, ten per class, held out from training. Included in the project; these are what Tier 1 demonstrates on. CC BY 4.0, A
data/sample_images/	Synthetic images used only by the automated test suite. Not a demo set and not real patient data.
results/	Every published number and chart: metrics.json, the comparison table, confusion matrices, ROC curves and training histories.
docs/	The finished documents: this guide, the project report, the paper and the technical reference.
docs/src/	The HTML and Markdown sources those documents are generated from.

Troubleshooting

- **No models found** — `--fetch-models` needs both `dist/models.zip.part00` and `part01`. If either is missing, the clone was incomplete; re-clone the repository.
- **SHA mismatch / CHECK FAIL** — a part file is truncated or was altered. Re-clone, or restore the two parts with `git checkout -- dist/`, then re-run `--fetch-models`.
- **Port in use** — `python run.py --port 8010`; add `--no-browser` to suppress the browser tab.
- **Browser tests** — `pip install -r requirements-dev.txt && playwright install chromium`.

12. Using the App — Step-by-Step Demo

This section walks through a complete demonstration of the application, suitable for a viva or client presentation (~10 minutes).

Before You Start

Nothing. After Tier 1 the Preview page already lists the 40 bundled images in `data/demo_images/` — ten per class, all held out from training — so every step below works on a fresh install.

If you have completed Tier 2 and want to demonstrate on the full held-out set instead, run `python run.py --cli export-test-images` once. That copies ten images per class into `test_images/`, which the dashboard prefers over the bundled set when present.

Step 1 — Start the App: Run python app.py. It opens http://localhost:8000 in your browser.

Step 2 — Log In: Enter admin / admin and press Login. A 'Login successful' toast appears and you land on Home.

Step 3 — Home: Show the paper title, citation, and the pipeline stepper (Data → Train → Ensemble → Evaluate → Predict). Point out the 'Not for clinical use' footer on every page.

Step 4 — Preview: Test Images: Open Preview. Under 'Try a test image', pick one from the Benign group. Its preview appears. Press Submit. The result shows the predicted class, confidence ring, probability bars, Grad-CAM overlay (drag the opacity slider) and per-backbone votes. Repeat for each malignant class.

Step 5 — Preview: Upload: Press 'Choose image' and pick any file. Then try dropping a .txt file to show the inline error message (no crash).

Step 6 — Performance Analysis: The four large tiles show Accuracy, Precision, Recall and F-measure. The confusion matrix is directly below. Click tabs to compare backbones. Scroll down for ROC curves, per-class metrics and the Grad-CAM grid.

Step 7 — Chart: Model Loss and Model Accuracy show train vs validation per epoch. Switch between backbones with the tabs.

Step 8 — About: Shows the architecture diagram, method summary, and disclaimer.

Step 9 — Settings: The sidebar switch 'Single model: MobileNetV2' makes Preview use one model alone. Switch it back to ensemble.

Step 10 — Logout: Press Logout. You return to the login screen; deep links require logging in again.

13. Glossary

Accuracy: The percentage of all predictions that were correct.

ALL (Acute Lymphoblastic Leukaemia): A type of blood cancer affecting white blood cells.

Augmentation: Creating altered copies of training images (flipped, rotated, etc.) to help the model generalise.

AUC (Area Under the Curve): A single number summarising how well the model separates classes; 1.0 is perfect.

Backbone: The core image-recognition neural network (e.g., MobileNetV2).

Batch size: The number of images processed together in one step of training (32 in this project).

Confusion matrix: A grid showing how many images of each true class were predicted as each class.

Convolution: A mathematical operation that slides a small filter across an image to detect patterns.

Dense connections: A design where every layer feeds into every later layer (used by DenseNet).

Ensemble: A method that combines several models to make a joint prediction.

Epoch: One complete pass through all training images.

F1-score: The harmonic mean of precision and recall; balances both measures.

Fine-tuning: Adjusting a pre-trained model's weights on a new, specific dataset.

Frozen layers: Layers whose weights are locked and do not change during training.

Grad-CAM: A technique that highlights which image regions influenced the model's prediction.

Heatmap: A coloured overlay showing regions of high (red) and low (blue) importance.

Hybrid: Using two different ensemble methods (voting and stacking) and selecting the better one.

ImageNet: A public collection of over 14 million labelled photographs used to pre-train models.

Kappa: Cohen's Kappa — a metric that measures agreement corrected for chance.

Learning rate: How large each adjustment step is during training; too large = unstable, too small = slow.

Logistic regression: A simple mathematical model that learns to classify based on input numbers.

Loss: A number measuring how wrong the model's prediction is; training aims to minimise it.

Macro-F1: The F1-score averaged equally across all classes.

Meta-classifier: A second-stage model that learns to combine the outputs of other models.

Neural network: A chain of mathematical operations, inspired by brain neurons, that learns patterns from data.

OOD (Out-of-Distribution): An image that looks very different from what the model was trained on.

Overfitting: When a model memorises training data rather than learning general patterns.

Peripheral blood smear: A thin layer of blood on a glass slide, stained and photographed under a microscope.

Precision: Of all images the model predicted as class X, what fraction actually were X.

Pre-trained: A model that has already learned from a large dataset before being adapted to a new task.

Recall: Of all images that truly are class X, what fraction the model correctly identified.

ROC curve: A plot of true-positive rate vs. false-positive rate across confidence thresholds.

Softmax: A function that converts raw model outputs into probabilities that sum to 1.

Stacking: An ensemble method where a second model learns to combine the first models' outputs.

Stratified split: Dividing data so each subset has the same class proportions as the whole.

Support: The number of test samples in a given class.

Transfer learning: Using knowledge from one task (ImageNet) to help with another (blood cells).

Validation set: A held-out portion of data used to tune the model during training.

Voting (weighted soft): An ensemble method that averages models' probabilities using weights.

Weights: The adjustable numbers inside a neural network that are learned during training.

14. References

[1] J. Jayachitra and N. Umarkathaf, "Blood Cancer Identification using Hybrid Ensemble Deep Learning Technique," 2023 International Conference on Emerging Research in Applied Sciences (ICEARS), IEEE, 2023.

[2] M. A. Eshraghi et al., "Blood Cells Cancer (ALL) — 4 Classes," Kaggle dataset.

[3] M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR 2018.

[4] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML 2019.

[5] G. Huang et al., "Densely Connected Convolutional Networks," CVPR 2017.

[6] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," ICCV 2017.

Disclaimer: This software is a research and educational project. It is not approved for clinical use. All predictions must be verified by a qualified medical professional.